DATABASE MANAGEMENT SYSTEMS

MINI PROJECT REPORT

Event Registration System

A Java and MySQL Application

Submitted by	Chandana H
Course	BCA (Hons.)
University	RV University
Subject	Database Management Systems

Submitted as part of the DBMS mini-project

1. Introduction

The Event Registration System was developed to make it easier to keep track of people registering for college events. The application has a Java Swing interface, while MySQL stores the registration details. JDBC is used to connect the application to the database.

Instead of maintaining participant details manually, the user can enter information through the form and view the saved records in the application. The records remain in the database after the application is closed.

2. Problem Statement

When registrations are handled manually, it can take time to find a participant's details. Updating information or removing an old record can also lead to mistakes. This project uses a MySQL database to keep registration details in one place and make them easier to manage.

3. Objectives

• Create a MySQL database and a table for event registrations.

• Connect the Java application to MySQL using JDBC.

• Add new participant details and display saved records.

• Allow existing registration details to be updated or deleted.

• Provide a simple interface for searching and managing records.

4. Database Design

The database is named **event_db**, and the main table is **registrations**. Each registration has a unique registration ID, which is generated automatically by MySQL.

Column	Type	Purpose
registration_id	INT	Unique ID; primary key; auto-increment
participant_name	VARCHAR(100)	Participant's name
email	VARCHAR(100)	Email address
event_name	VARCHAR(100)	Name of the event
registration_date	DATE	Date of registration
usn	VARCHAR(30)	University seat number
course	VARCHAR(50)	Course name
semester	INT	Current semester

The primary key helps identify each record separately. The participant name, email, event name, and registration date are required fields in the table definition.

5. SQL Queries

5.1 Create the database and table

```
CREATE DATABASE IF NOT EXISTS event_db;
USE event_db;

CREATE TABLE IF NOT EXISTS registrations (
    registration_id INT PRIMARY KEY AUTO_INCREMENT,
    participant_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    event_name VARCHAR(100) NOT NULL,
    registration_date DATE NOT NULL,
    usn VARCHAR(30),
    course VARCHAR(50),
    semester INT
);
```

5.2 Insert a registration

```
INSERT INTO registrations
(participant_name, email, event_name, registration_date,
 usn, course, semester)
VALUES
('Sample Student', 'student@example.com', 'Technical Fest',
 '2026-10-27', '1RUA25BCA0001', 'BCA', 2);
```

This is a sample INSERT query. For the demonstration, use a record entered through the application.

5.3 View registrations

```
SELECT * FROM registrations;
```

5.4 Update a registration

```
UPDATE registrations
SET event_name = 'Coding Workshop'
WHERE registration_id = 1;
```

This query changes the event name for the record with ID 1, if that record exists.

5.5 Delete a registration

```
DELETE FROM registrations
WHERE registration_id = 1;
```

This removes the record with ID 1. Run it only when you intend to delete that record.

6. Application and Database Connection

The front end is built with Java Swing. It contains fields for participant details and a table for viewing registrations. The user can add a registration, search for records, update selected details, or delete a record.

MySQL is used to store the data, and JDBC passes SQL commands from the Java application to the database. The MySQL Connector/J driver is included in the project so that Java can connect to MySQL.

```
String url = "jdbc:mysql://localhost:3306/event_db";
String user = "root";
String password = "YOUR_MYSQL_PASSWORD";

Connection con =
    DriverManager.getConnection(url, user, password);
```

In the actual source code, the password must be replaced with the password configured for the local MySQL server.

7. Database Operations

Insert: Saves a new participant's registration details.

Select: Retrieves saved records and displays them in the application.

Update: Changes the details of an existing registration.

Delete: Removes a selected registration from the database.

Search: Helps the user find a registration using the available search options.

8. Conclusion

This project provided practical experience in connecting a Java application to a MySQL database. The Event Registration System stores participant details in a table and provides a way to add, view, search, update, and delete registration records.

The project uses a primary key to identify each registration and JDBC to send database queries from the application. It shows how SQL and a simple user interface can work together to manage information. In the future, the system could be extended with login access, email confirmations, or reports for event organisers.